Aizat Taqqiyudin

AI Engineer | Python and TypeScript | RAG, LLM evaluation and generative AI applications

aizattaqq@gmail.com | +6017 284 0608 | Putrajaya, Malaysia | github.com/ixtaqq | linkedin.com/in/aizattaqq

SUMMARY

Software engineer building generative AI and LLM applications in Python and TypeScript. My main project answers questions about SEC filings with cited passages and is served over MCP. It has a dev/test evaluation set and a CI check that fails when retrieval quality drops. I also built and use two Telegram briefing services, and I have a background in web development and UI design. I'm looking for AI Engineer, Applied AI Engineer or LLM application roles.

PROJECTS

Goldirham Filings | Cited Q&A over SEC filings, served over MCP | Python | Oct 2026
github.com/ixtaqq/goldirham-filings

- Built a Retrieval-Augmented Generation (RAG) system, served as a Model Context Protocol (MCP) server, that answers questions about 96 10-K and 10-Q filings from 12 AI-infrastructure companies. Every answer sentence cites a filing passage, or the system says the filings don't answer the question. The server exposes five read-only tools for coverage, search, passage lookup, answers and filing comparison.
- Wrote a rate-limited EDGAR client with retries, an inline-XBRL parser that splits filings by Item heading, and section-aware chunking. Hypothesis property tests check that every chunk's offsets map back to its exact source text.
- Retrieval merges Postgres full-text search and pgvector semantic search with reciprocal rank fusion. Fiscal-period scoping resolves phrases like "most recent 10-Q", so a question about a quarter that isn't indexed gets an abstention instead of another quarter's numbers. Questions that name several filings search each one separately, so one long filing can't crowd out the others.
- Guardrails in code check every answer. They drop citations to passages the model never received, remove uncited sentences and flag numbers missing from the cited passages. Everything runs locally on Ollama, so there is no API cost.
- Built an evaluation harness on a 68-question golden set with a stratified dev/test split (Recall@10, MRR, citation precision, abstention rate) and logged every change as kept or reverted. A prompt rule against computing new figures cut answers with unsupported numbers from 3 to 1. Rejected a cross-encoder reranker because MRR rose only 0.5 points (the bar was 3), Recall@10 fell from 0.740 to 0.677, and each query took 3.5 s longer.
- A CI retrieval gate on a committed fixture corpus fails any change that lowers Recall@10 or MRR by more than 0.02; removing vector search dropped Recall@10 from 0.769 to 0.654 and failed the build.

Tech: Python, MCP, PostgreSQL, pgvector, Ollama (qwen3, nomic-embed-text), SQLAlchemy, Alembic, Pydantic, pytest, Hypothesis, mypy (strict), ruff, Docker, GitHub Actions

Goldirham Stack | Personal AI-infrastructure news and filings digest | TypeScript | Jun 2026 – Present
goldirham-stack.vercel.app | github.com/ixtaqq/ai-infra-digest

- Built and run this pipeline for my own work. Each day it pulls 68 RSS feeds and the 8-K, 10-K and 10-Q filings of 35 companies, classifies news into 10 sectors, and sends a personalised Telegram digest at a set local time.
- Split model work by cost. A free keyword filter and a fast model (about $0.0005 per filing) decide which SEC filings matter, and a strong model (about $0.01 each) extracts numbers from those only. The same fast/strong split handles news classification and synthesis, and responses are cached by input hash so a re-run doesn't call the model again.
- Validated every model response with a Zod schema and passed untrusted article text to the model as escaped JSON to reduce prompt-injection risk. Optional stages fail on their own without stopping the pipeline.
- Deduplicates by URL and Jaccard similarity; optional pgvector embedding matching is off in production.
- Built idempotent per-user delivery with atomic claims, jittered retries, and optional Slack and email copies. The Supabase backend has 41 migrations, row-level security on every table and automated data retention.
- CI runs lint, 630+ Vitest unit and integration tests, migration checks, a dependency audit and blocking Semgrep.

Tech: TypeScript, Node.js, Supabase (Postgres, pgvector), Groq and OpenAI APIs, Zod, Vitest, Docker, GitHub Actions, Render, Vercel

Goldirham Lens | Daily market research briefing | Python | Oct 2026
goldirham-lens.vercel.app

- A Python service on GitHub Actions and Vercel, with no server to maintain. Each morning it collects crypto, US and Bursa Malaysia market data, computes indicators, setups and portfolio state, and sends one Telegram briefing before 08:00 Malaysia time.
- An optional summary from Claude cannot add numbers of its own; every figure comes from the analysis code.
- Analysis and composition are pure functions of one input snapshot, so any briefing can be rebuilt byte for byte from its stored inputs, and backtests reuse the production decision code.
- Two scheduled triggers and one database claim per day make a missed or doubled briefing unlikely. A dead-man's switch reports missed runs, a reduced briefing goes out when the database is down, and a delivery state machine holds partly sent messages instead of sending them twice.
- 260+ tests, including PostgreSQL integration and golden-file tests; strict typing, import contracts, automated backups with restore drills, and 12 architecture decision records.

Tech: Python, FastAPI, SQLAlchemy (async), Alembic, pandas, Anthropic API, pytest, Hypothesis, mypy, Vercel, Supabase, Next.js

Goldirham Insight | Investment research site | TypeScript | Jun 2026 – Present
goldirham-insight.vercel.app | github.com/ixtaqq/Goldirham-Insight

- Next.js research site covering 234 assets across AI infrastructure, growth funds and crypto, with 26 scored assets, live crypto quotes and simulated stock data. Each price is labelled as coming from a market feed or a simulation.

Tech: Next.js, React, TypeScript (strict), Tailwind CSS, Playwright, Vercel

EXPERIENCE

UI/UX Designer | GFIS (M) Sdn. Bhd. | 09/2022 – 11/2024

- Developed an internal web application alongside the main development team.
- Designed and built the admin dashboard for an internal project.
- Built interactive interface components (forms, buttons, animations) and deployed builds to Vercel for testing.
- Designed accessible interfaces that followed the company brand, working with the developers.

Junior Software Developer (Internship) | GFIS Sdn. Bhd. | 06/2022 – 09/2022

- Wrote, tested and debugged web application code with senior developers using Next.js, Strapi, MongoDB and Django.
- Helped design and build user interfaces.

TECHNICAL SKILLS

AI and LLMs: Generative AI, Retrieval-Augmented Generation (RAG), hybrid and semantic search, vector databases (pgvector), embeddings, reciprocal rank fusion, chunking, LLM retrieval and answer evaluation (Recall@k, MRR, citation precision, abstention), prompt engineering, guardrails, Model Context Protocol (MCP) servers, structured output with Pydantic and Zod, multi-model routing and cost control, prompt-injection mitigation

Model providers: Anthropic, OpenAI, Groq, OpenRouter, Ollama (local models)

Languages: Python, TypeScript, JavaScript, SQL

Backend and data: PostgreSQL, pgvector, Supabase, SQLAlchemy, Alembic, FastAPI, Node.js, pandas, REST APIs, webhooks

Testing and DevOps: pytest, Hypothesis, Vitest, Playwright, mypy, ruff, CI/CD (GitHub Actions), Docker, Semgrep, Vercel, Render

Frontend: Next.js, React, Tailwind CSS, UI/UX design

EDUCATION

Bachelor of Computer Science (Software Engineering) | Universiti Tenaga Nasional (UNITEN) | 2019 – 2022
CGPA 3.4 | Dean's List (Semester 1)

Foundation in Computer Science | Universiti Tenaga Nasional (UNITEN) | 2018 – 2019

CERTIFICATIONS

Introduction to Model Context Protocol, Anthropic | Claude 101, Anthropic